

# C++ Programming

## Day 9: 템플릿

함수 템플릿, 다중 타입, 오버로딩, 클래스 템플릿, 기본 템플릿 인자

## Contents

|           |                          |           |
|-----------|--------------------------|-----------|
| <b>1</b>  | <b>학습 목표</b>             | <b>2</b>  |
| <b>2</b>  | <b>템플릿의 필요성</b>          | <b>2</b>  |
| <b>3</b>  | <b>함수 템플릿</b>            | <b>3</b>  |
| 3.1       | 기본 문법                    | 3         |
| 3.2       | typename과 class          | 3         |
| 3.3       | 자료형 추론                   | 3         |
| 3.4       | 같은 T가 의미하는 것             | 4         |
| 3.5       | 전체 예제                    | 4         |
| <b>4</b>  | <b>여러 개의 템플릿 타입 매개변수</b> | <b>4</b>  |
| 4.1       | 독립적인 자료형 추론              | 4         |
| 4.2       | 전체 예제                    | 5         |
| 4.3       | 타입 매개변수의 개수              | 5         |
| <b>5</b>  | <b>일반 함수와 템플릿 함수</b>     | <b>6</b>  |
| 5.1       | 정확한 일치와 형 변환             | 6         |
| 5.2       | 선택 원리                    | 6         |
| 5.3       | 전체 예제                    | 6         |
| 5.4       | double 일반 함수와의 비교        | 7         |
| <b>6</b>  | <b>클래스 템플릿</b>           | <b>7</b>  |
| 6.1       | 클래스 중복 문제                | 7         |
| 6.2       | 객체 생성 시 자료형 지정           | 8         |
| 6.3       | 생성자와 멤버 함수               | 8         |
| 6.4       | 전체 예제                    | 9         |
| 6.5       | STL과의 연결                 | 9         |
| <b>7</b>  | <b>기본 템플릿 인자</b>         | <b>10</b> |
| 7.1       | 기본 문법                    | 10        |
| 7.2       | 전체 예제                    | 10        |
| 7.3       | 템플릿 인자 결정과 형 변환          | 11        |
| <b>8</b>  | <b>종합 정리</b>             | <b>12</b> |
| <b>9</b>  | <b>개념별 실습 완성 코드</b>      | <b>12</b> |
| 9.1       | 함수 템플릿                   | 12        |
| 9.2       | 여러 개의 템플릿 타입             | 13        |
| 9.3       | 일반 함수와 템플릿 함수            | 13        |
| 9.4       | 클래스 템플릿                  | 14        |
| 9.5       | 기본 템플릿 인자                | 14        |
| <b>10</b> | <b>실습 파일 구성</b>          | <b>15</b> |

## 1. 학습 목표

### 학습 목표

이 장을 마치면 다음 내용을 설명하고 직접 코드로 작성할 수 있다.

- 코드 중복이 발생하는 이유와 템플릿의 필요성 설명하기
- `template <typename T>` 문법으로 함수 템플릿 작성하기
- 호출 인수로부터 템플릿 자료형이 추론되는 과정 설명하기
- 여러 개의 템플릿 타입 매개변수를 동시에 사용하기
- 일반 함수와 템플릿 함수가 함께 있을 때 호출 우선순위 이해하기
- `template <typename T>` 문법으로 클래스 템플릿 작성하기
- 객체를 만들 때 클래스 템플릿의 실제 자료형 지정하기
- 기본 템플릿 인자를 선언하고 생략 문법 사용하기
- 템플릿 자료형 결정과 암시적 형 변환을 구분하기
- 템플릿이 이후 STL 컨테이너의 기반이 되는 이유 설명하기

## 2. 템플릿의 필요성

자료형만 다르고 동작은 같은 함수나 클래스를 여러 번 작성하면 코드 중복이 발생한다. 다음 두 함수는 매개변수와 반환형만 다르고 수행하는 연산은 같다.

```

1 int add(int a, int b)
2 {
3     return a + b;
4 }
5
6 double add(double a, double b)
7 {
8     return a + b;
9 }
```

자료형이 `float`, `long`, `string`으로 늘어나면 같은 형태의 코드를 계속 반복해야 한다. 템플릿(`template`)은 자료형을 미리 고정하지 않고, 실제 호출이나 객체 생성 시점에 필요한 자료형을 컴파일러가 결정하도록 만드는 기능이다.

Repeated code

```

int add(int, int)
double add(double, double)
float add(float, float)
```

|  
v

One template

```
T add(T, T)
```

템플릿은 실행 중에 자료형이 바뀌는 기능이 아니다. 컴파일러가 코드를 번역할 때 실제로 사용된 자료형에 맞는 함수 또는 클래스를 생성한다.

**핵심 포인트**

템플릿은 자료형에 의존하지 않는 공통 알고리즘을 한 번만 작성하게 한다. 실제로 사용되는 자료형별 코드는 컴파일 시점에 생성된다.

### 3. 함수 템플릿

함수 템플릿(function template)은 자료형만 다른 여러 함수를 하나의 일반화된 형태로 작성하는 기능이다.

#### 3.1. 기본 문법

함수 선언 앞에 다음 문장을 작성한다.

```
1 template <typename T>
```

T는 아직 결정되지 않은 자료형을 나타내는 템플릿 타입 매개변수(template type parameter)이다. 이름은 반드시 T일 필요는 없지만, 관례적으로 첫 번째 타입에 T를 자주 사용한다.

```
1 template <typename T>
2 T add(T a, T b)
3 {
4     return a + b;
5 }
```

위 코드에서 T는 반환형, 첫 번째 매개변수형, 두 번째 매개변수형에 모두 사용된다.

#### 3.2. typename과 class

템플릿 타입 매개변수 선언에는 다음 두 문법을 사용할 수 있다.

```
1 template <typename T>
```

```
1 template <class T>
```

기본적인 타입 매개변수 선언에서는 두 표현의 의미가 같다. 현대 C++ 코드에서는 해당 매개변수가 자료형임을 더 직접적으로 나타내는 `typename`을 자주 사용한다.

#### 3.3. 자료형 추론

함수 템플릿을 호출하면 컴파일러가 전달된 인수의 자료형을 보고 T를 추론한다.

```
1 add(10, 20);
```

두 인수가 모두 `int`이므로 컴파일러는 개념적으로 다음 함수를 만든다.

```
1 int add(int a, int b)
2 {
3     return a + b;
4 }
```

다음 호출에서는 T가 `double`이 된다.

```
1 add(2.5, 3.1);
```

```
1 double add(double a, double b)
2 {
3     return a + b;
4 }
```

이처럼 하나의 템플릿 정의로 여러 자료형에 대응하는 함수를 생성할 수 있다.

### 3.4. 같은 T가 의미하는 것

다음 템플릿에서 두 매개변수는 모두 같은 T를 사용한다.

```
1 template <typename T>
2 T multiply(T a, T b)
```

따라서 다음 호출은 자연스럽게 추론된다.

```
1 multiply(3, 4);
2 multiply(2.5, 4.2);
```

그러나 다음 호출에서는 첫 번째 인수가 int, 두 번째 인수가 double이므로 하나의 T를 결정할 수 없다.

```
1 multiply(3, 2.5);
```

이 문제는 서로 다른 타입 매개변수를 두 개 선언하면 해결할 수 있다.

### 3.5. 전체 예제

```
1 #include <iostream>
2
3 using namespace std;
4
5 template <typename T>
6 T add(T a, T b)
7 {
8     return a + b;
9 }
10
11 int main()
12 {
13     cout << add(10, 20) << endl;
14     cout << add(2.5, 3.1) << endl;
15
16     return 0;
17 }
```

출력 결과는 다음과 같다.

```
30
5.6
```

## 4. 여러 개의 템플릿 타입 매개변수

서로 다른 자료형을 동시에 받아야 한다면 템플릿 타입 매개변수를 여러 개 선언한다.

```
1 template <typename T, typename U>
```

T와 U는 서로 독립적인 자료형이다. 첫 번째 인수에서 T를 추론하고 두 번째 인수에서 U를 추론할 수 있다.

### 4.1. 독립적인 자료형 추론

다음 호출을 살펴보자.

```
1 show(100, 3.14);
```

컴파일러는 다음과 같이 추론한다.

```
T = int
U = double
```

다음 호출에서는 다른 조합이 만들어진다.

```
1 show("Age", 20);
```

문자열 리터럴은 함수 매개변수로 전달될 때 일반적으로 문자열을 가리키는 포인터 형태로 처리되므로 개념적으로 다음과 같은 조합이 된다.

```
T = const char*
U = int
```

## 4.2. 전체 예제

```
1 #include <iostream>
2
3 using namespace std;
4
5 template <typename T, typename U>
6 void show(T first, U second)
7 {
8     cout << first << " " << second << endl;
9 }
10
11 int main()
12 {
13     show(100, 3.14);
14     show("Age", 20);
15
16     return 0;
17 }
```

출력 결과는 다음과 같다.

```
100 3.14
Age 20
```

## 4.3. 타입 매개변수의 개수

필요하다면 세 개 이상의 타입 매개변수도 선언할 수 있다.

```
1 template <typename T, typename U, typename V>
2 void printThree(T first, U second, V third)
3 {
4     cout << first << " " << second << " " << third << endl;
5 }
```

다만 타입 매개변수가 많아질수록 함수의 사용 조건과 반환형 설계가 복잡해질 수 있으므로 실제 필요에 맞게 사용한다.

### 핵심 포인트

같은 이름의 타입 매개변수는 같은 자료형을 의미한다. 서로 다른 자료형을 독립적으로 받으려면 T, U처럼 별도의 타입 매개변수를 선언해야 한다.

## 5. 일반 함수와 템플릿 함수

일반 함수와 함수 템플릿은 같은 이름으로 함께 존재할 수 있다. 이때 컴파일러는 함수 오버로딩 규칙에 따라 가장 적합한 후보를 선택한다.

### 5.1. 정확한 일치와 형 변환

다음 두 함수가 있다고 하자.

```
1 void print(int x)
2 {
3     cout << "Normal Function" << endl;
4 }
5
6 template <typename T>
7 void print(T x)
8 {
9     cout << "Template Function" << endl;
10 }
```

`print(10)`은 일반 함수 `print(int)`와 정확히 일치한다. 템플릿으로도 `print(int)`를 만들 수 있지만, 동일하게 정확한 후보라면 일반 함수가 선택된다.

```
1 print(10);
```

Normal Function

반면 `print(3.14)`에서 일반 함수는 `double`을 `int`로 바꾸는 형 변환이 필요하다. 템플릿은 `T = double`로 정확히 일치하는 함수를 만들 수 있으므로 템플릿 함수가 선택된다.

```
1 print(3.14);
```

Template Function

### 5.2. 선택 원리

함수 선택은 단순히 “일반 함수가 항상 우선”이라고 이해하면 안 된다. 먼저 각 후보가 인수와 얼마나 잘 일치하는지 비교한다.

| 상황                          | 선택 결과  |
|-----------------------------|--------|
| 일반 함수만 정확히 일치               | 일반 함수  |
| 템플릿 함수만 정확히 일치              | 템플릿 함수 |
| 둘 다 정확히 일치                  | 일반 함수  |
| 일반 함수는 형 변환 필요, 템플릿은 정확히 일치 | 템플릿 함수 |

### 5.3. 전체 예제

```
1 #include <iostream>
2
3 using namespace std;
4
5 void print(int x)
6 {
7     cout << "Normal Function" << endl;
8 }
9
```

```

10 template <typename T>
11 void print(T x)
12 {
13     cout << "Template Function" << endl;
14 }
15
16 int main()
17 {
18     print(10);
19     print(3.14);
20
21     return 0;
22 }

```

출력 결과는 다음과 같다.

Normal Function

Template Function

## 5.4. double 일반 함수와의 비교

다음 코드에서는 첫 번째 호출과 두 번째 호출의 선택 결과가 다르다.

```

1 void print(double x)
2 {
3     cout << "Double Function" << endl;
4 }
5
6 template <typename T>
7 void print(T x)
8 {
9     cout << "Template Function" << endl;
10 }

```

print(3.14)은 일반 함수가 정확히 일치하므로 Double Function이 출력된다. print(100)은 일반 함수에서 int를 double로 바꿔야 하지만, 템플릿은 T = int로 정확히 일치하므로 Template Function이 출력된다.

### 핵심 포인트

정확한 일치 여부가 먼저 비교된다. 일반 함수와 템플릿 함수가 모두 같은 수준으로 정확히 일치할 때 일반 함수가 선택된다.

## 6. 클래스 템플릿

클래스 템플릿(class template)은 멤버 변수나 멤버 함수가 사용할 자료형을 객체 생성 시점까지 미루는 기능이다.

### 6.1. 클래스 중복 문제

값 하나를 저장하는 클래스를 자료형별로 만들면 다음과 같이 코드가 반복된다.

```

1 class IntBox
2 {
3 public:
4     int value;
5 };
6

```



```

7 class DoubleBox
8 {
9 public:
10     double value;
11 };

```

클래스 템플릿을 사용하면 하나의 클래스 정의로 여러 자료형을 처리할 수 있다.

```

1 template <typename T>
2 class Box
3 {
4 public:
5     T value;
6 };

```

## 6.2. 객체 생성 시 자료형 지정

함수 템플릿은 인수로부터 타입을 추론할 수 있지만, 기본적인 클래스 템플릿 사용에서는 객체를 만들 때 각괄호 안에 실제 자료형을 명시한다.

```

1 Box<int> box1;
2 Box<double> box2;
3 Box<char> box3;

```

Box<int>와 Box<double>은 같은 템플릿에서 만들어지지만 서로 다른 구체 클래스 타입이다.

```

Box<T>
|
+-- Box<int>
+-- Box<double>
+-- Box<char>

```

## 6.3. 생성자와 멤버 함수

클래스 내부에서는 템플릿 타입 매개변수 T를 멤버 변수, 생성자 매개변수, 반환형 등 필요한 위치에 사용할 수 있다.

```

1 template <typename T>
2 class Box
3 {
4 public:
5     T value;
6
7     Box(T value)
8     {
9         this->value = value;
10    }
11
12    void show()
13    {
14        cout << value << endl;
15    }
16 };

```

Box<int>를 만들면 T value와 Box(T value)의 T가 모두 int로 바뀐다. Box<double>에서는 모두 double로 바뀐다.

## 6.4. 전체 예제

```

1 #include <iostream>
2
3 using namespace std;
4
5 template <typename T>
6 class Box
7 {
8 public:
9     T value;
10
11     Box(T value)
12     {
13         this->value = value;
14     }
15
16     void show()
17     {
18         cout << value << endl;
19     }
20 };
21
22 int main()
23 {
24     Box<int> box1(100);
25     Box<double> box2(3.14);
26     Box<char> box3('A');
27
28     box1.show();
29     box2.show();
30     box3.show();
31
32     return 0;
33 }

```

출력 결과는 다음과 같다.

```

100
3.14
A

```

## 6.5. STL과의 연결

표준 템플릿 라이브러리(STL)의 컨테이너는 클래스 템플릿으로 구현되어 있다. 다음 문법의 각괄호 안 자료형은 클래스 템플릿의 타입 인자이다.

```

1 vector<int> numbers;
2 vector<double> values;
3 stack<char> characters;
4 map<string, int> scores;

```

따라서 클래스 템플릿을 이해하면 이후 vector, list, map, stack, queue 등의 문법을 자연스럽게 해석할 수 있다.

### 핵심 포인트

클래스 템플릿은 하나의 클래스가 여러 자료형을 저장하거나 처리하도록 만든다. Box<int>와 Box<double>은 같은 템플릿에서 생성되지만 서로 다른 타입이다.

## 7. 기본 템플릿 인자

기본 템플릿 인자(default template argument)는 사용자가 자료형을 명시하지 않았을 때 사용할 기본 자료형을 지정하는 기능이다.

### 7.1. 기본 문법

다음 템플릿은 T의 기본 자료형을 int로 지정한다.

```
1 template <typename T = int>
2 class Number
3 {
4 };
```

객체를 만들 때 빈 각괄호를 사용하면 기본 자료형이 적용된다.

```
1 Number<> n1;
```

위 코드는 다음 코드와 같은 의미이다.

```
1 Number<int> n1;
```

직접 자료형을 명시하면 기본값은 사용되지 않는다.

```
1 Number<double> n2;
```

### 7.2. 전체 예제

```
1 #include <iostream>
2
3 using namespace std;
4
5 template <typename T = int>
6 class Number
7 {
8 public:
9     T value;
10
11     Number(T value)
12     {
13         this->value = value;
14     }
15
16     void show()
17     {
18         cout << value << endl;
19     }
20 };
21
22 int main()
23 {
24     Number<> n1(100);
25     Number<double> n2(3.14);
26
27     n1.show();
28     n2.show();
29
30     return 0;
31 }
```

출력 결과는 다음과 같다.

100  
3.14

### 7.3. 템플릿 인자 결정과 형 변환

다음 클래스에서 기본 자료형은 double이다.

```
1 template <typename T = double>
2 class Value
3 {
4 public:
5     T value;
6
7     Value(T value)
8     {
9         this->value = value;
10    }
11};
```

다음 객체 생성에서 전달한 10은 int 리터럴이다.

```
1 Value<> v1(10);
```

그러나 처리 순서는 다음과 같다.

1. Value<> uses the default template argument.
2. T becomes double.
3. The constructor becomes Value(double value).
4. The int value 10 is converted to double.
5. The member stores 10.0.

즉, 10의 자료형을 보고 T를 다시 int로 정하는 것이 아니다. Value<>에서 기본 템플릿 인자가 먼저 적용되어 Value<double>이 되고, 그 다음 생성자 호출 과정에서 int가 double로 암시적 형 변환된다. 반대 방향도 문법적으로 가능하다.

```
1 Value<int> v2(3.14);
```

이 경우 T는 명시적으로 int이며, 3.14가 int로 변환되어 소수 부분이 제거된 3이 저장된다.

| 코드                  | 결정된 T  | 생성자 전달 과정        |
|---------------------|--------|------------------|
| Value<> v1(10)      | double | int에서 double로 변환 |
| Value<char> v2('Z') | char   | 정확히 일치           |
| Value<int> v3(3.14) | int    | double에서 int로 변환 |

#### 핵심 포인트

템플릿 자료형을 결정하는 단계와 생성자 인수의 형 변환 단계는 서로 다르다. Value<>에서는 기본 템플릿 인자가 먼저 적용되고, 그 뒤 전달값이 생성자의 매개변수형에 맞게 변환된다.

## 8. 종합 정리

### 핵심 포인트

- 템플릿은 자료형만 다른 중복 코드를 하나의 일반화된 코드로 작성하게 한다.
- 템플릿은 런타임 기능이 아니라 컴파일 시점에 필요한 코드를 생성하는 기능이다.
- 함수 템플릿은 `template <typename T>` 뒤에 함수를 정의한다.
- 함수 호출 인수로부터 템플릿 타입을 자동 추론할 수 있다.
- 같은 T를 사용한 여러 매개변수는 같은 자료형으로 추론되어야 한다.
- 서로 다른 자료형을 받으려면 T, U처럼 여러 타입 매개변수를 선언한다.
- 일반 함수와 템플릿 함수의 선택에서는 정확한 일치 여부가 중요하다.
- 일반 함수와 템플릿 함수가 모두 정확히 일치하면 일반 함수가 선택된다.
- 클래스 템플릿은 객체 생성 시 실제 자료형을 각괄호 안에 지정한다.
- 기본 템플릿 인자는 자료형을 생략했을 때 사용할 기본 자료형을 정한다.
- 템플릿 타입 결정과 생성자 인수의 암시적 형 변환은 별개의 단계이다.
- STL의 컨테이너는 클래스 템플릿이므로 Day 9의 내용이 STL 학습의 기반이 된다.

## 9. 개념별 실습 완성 코드

### 9.1. 함수 템플릿

#### 실습

두 값을 곱하여 반환하는 함수 템플릿 `multiply()`를 작성한다. `int`와 `double` 자료형으로 각각 호출한다.

```

1 #include <iostream>
2
3 using namespace std;
4
5 template <typename T>
6 T multiply(T a, T b)
7 {
8     return a * b;
9 }
10
11 int main()
12 {
13     cout << multiply(3, 4) << endl;
14     cout << multiply(2.5, 4.2) << endl;
15
16     return 0;
17 }
```

## 9.2. 여러 개의 템플릿 타입

### 실습

서로 다른 두 자료형의 값을 한 줄에 출력하는 함수 템플릿 `printPair()`를 작성한다.

```

1 #include <iostream>
2
3 using namespace std;
4
5 template <typename T, typename U>
6 void printPair(T first, U second)
7 {
8     cout << first << " " << second << endl;
9 }
10
11 int main()
12 {
13     printPair("Kim", 90);
14     printPair(3.5, 'A');
15
16     return 0;
17 }
```

## 9.3. 일반 함수와 템플릿 함수

### 실습

`double` 일반 함수와 템플릿 함수를 같은 이름으로 작성하고, `double`과 `int` 인수에서 어떤 함수가 선택되는지 확인한다.

```

1 #include <iostream>
2
3 using namespace std;
4
5 void print(double x)
6 {
7     cout << "Double Function" << endl;
8 }
9
10 template <typename T>
11 void print(T x)
12 {
13     cout << "Template Function" << endl;
14 }
15
16 int main()
17 {
18     print(3.14);
19     print(100);
20
21     return 0;
22 }
```

## 9.4. 클래스 템플릿

### 실습

하나의 값을 저장하고 출력하는 클래스 템플릿 Data를 작성한다. string과 double 자료형으로 객체를 생성한다.

```
1 #include <iostream>
2 #include <string>
3
4 using namespace std;
5
6 template <typename T>
7 class Data
8 {
9 public:
10     T value;
11
12     Data(T value)
13     {
14         this->value = value;
15     }
16
17     void show()
18     {
19         cout << value << endl;
20     }
21 };
22
23 int main()
24 {
25     Data<string> data1("Physics");
26     Data<double> data2(4.24);
27
28     data1.show();
29     data2.show();
30
31     return 0;
32 }
```

## 9.5. 기본 템플릿 인자

### 실습

기본 타입이 double인 클래스 템플릿 Value를 작성한다. 기본 타입과 직접 지정한 char 타입으로 객체를 생성한다.

```
1 #include <iostream>
2
3 using namespace std;
4
5 template <typename T = double>
6 class Value
7 {
8 public:
9     T value;
10 }
```

```
11 Value(T value)
12 {
13     this->value = value;
14 }
15
16 void show()
17 {
18     cout << value << endl;
19 }
20 };
21
22 int main()
23 {
24     Value<> v1(10);
25     Value<char> v2('Z');
26
27     v1.show();
28     v2.show();
29
30     return 0;
31 }
```

## 10. 실습 파일 구성

### 실습

Day 9 파일은 다음과 같이 관리한다.

- 01\_function\_template.cpp
- 02\_multiple\_types.cpp
- 03\_template\_overloading.cpp
- 04\_class\_template.cpp
- 05\_template\_default.cpp
- practice/p01\_function\_template.cpp
- practice/p02\_multiple\_types.cpp
- practice/p03\_template\_overloading.cpp
- practice/p04\_class\_template.cpp
- practice/p05\_template\_default.cpp